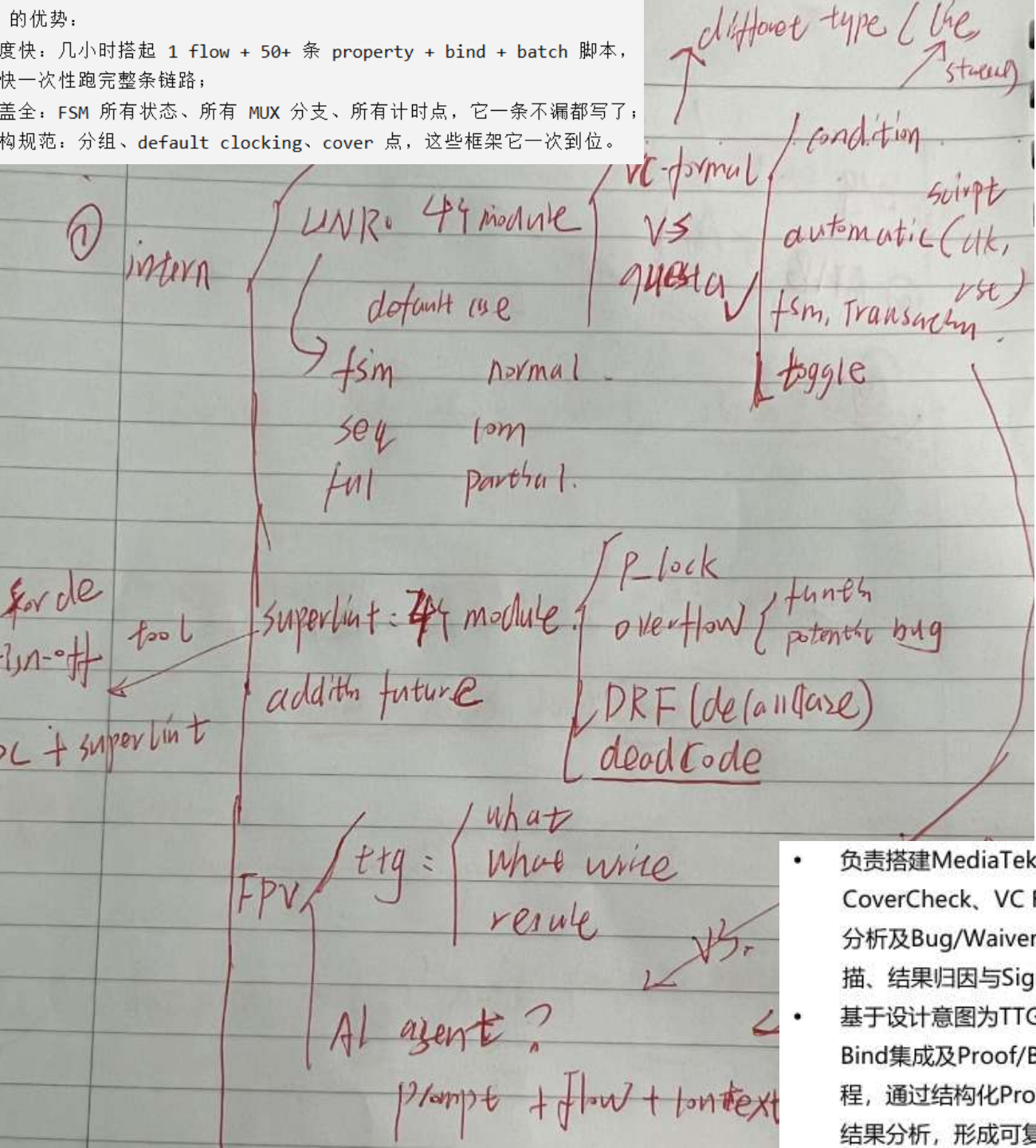
的优势：

度快：几小时搭起 1 flow + 50+ 条 property + bind + batch 脚本，

快一次性跑完整条链路；

盖全：FSM 所有状态、所有 MUX 分支、所有计时点，它一条不漏都写了；

构规范：分组、default clocking、cover 点，这些框架它一次到位。



4. 给我一个普通 module，是怎么开始写 FPV

1. 先看 RTL，别看 FSM 几个状态，哪些 MUX，几级 pipeline，哪些寄存器是某一时钟什么；
2. 先写 CEX 最容易的 property，把 property 和 baseline 先从初始环境里跑起来；
3. 再写所有的状态都有的接口；
4. 对 cover point：把所有的 cover 都跑一遍，确保不是因为 spec 定义不完整导致不合理的覆盖回来的每个条件；
5. 加 assumption：假设环境条件；
6. 加 cover：让 cover 能够逐渐展开；
7. 视一些 CEX 分类原因，CEX 分为三类—RTL bug、property 写法问题、assumption 不够。你先理解。

Ai_p

局部边界：CEX 会用 1 -> 当时应该是，但 delay_cnt 计数回绕，
复位刚释放第一拍这些边界，它没想到。

优先级冲突：RTL 里 rxiqc > ifcnq 的 else-if 优先级，
它写在 property 里加 rxiqc 后就有（sdm_frac/int_update_ifcnq 两条）。
Spec 的工程含义：ckref 要求，平常只包住 calib，
它写了时序断言但漏了复位弱一点的延迟逻辑
(kband_ckref 两条 CEX)。

输入约束不足：它自动加 assumption，但约束 autok_en、init_time
这类输入的合法范围，工具就会反例。

- 负责搭建MediaTek Wi-Fi 射频数字IP的Formal-based RTL与Coverage Sign-off支持流程，基于Questa CoverCheck、VC Formal UNR及Cadence SuperLint完成4个模块试点，开展Coverage Hole可达性判定、CEX根因分析及Bug/Waiver分类；通过跨工具Correlation沉淀规则与诊断知识库，并据此搭建UNR专项AI Agent，实现批量扫描、结果归因与Sign-off报告自动生成。
- 基于设计意图为TTG模块做FPV（形式验证），构建包含80+条Assume、Assert与Cover的SVA Property Set，完成Bind集成及Proof/Bounded Proof/CEX分析；以人工Property作为Golden Reference，设计Agent-assisted FPV流程，通过结构化Prompt、RTL与领域知识Context注入及工具调用链，驱动SVA、Bind与Tcl脚本生成、Formal执行及结果分析，形成可复用、可迭代的Formal验证自动化方案。

点	AHB	AXI
核心特点	地址和数据可以流水化	读写完全分离、多个通道独立握手
握手	主要看 `HREADY`	每个通道都是 `VALID && READY`
读写	共用一套总线传输逻辑	读、写可同时进行
性能	较高，但相对简单	更高，适合 CPU、DDR、DMA
验证重点	地址/数据流水关系、`HREADY` 插入等待、Burst	五通道握手、读写乱序、Burst、ID 对应关系

AXI 五通道只需要背名字：

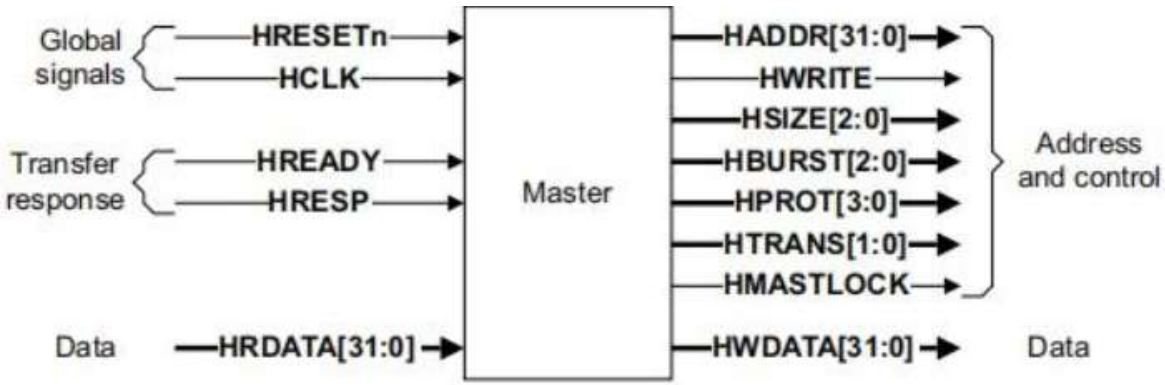
- `AW`：写地址
- `W`：写数据
- `B`：写响应
- `AR`：读地址
- `R`：读数据

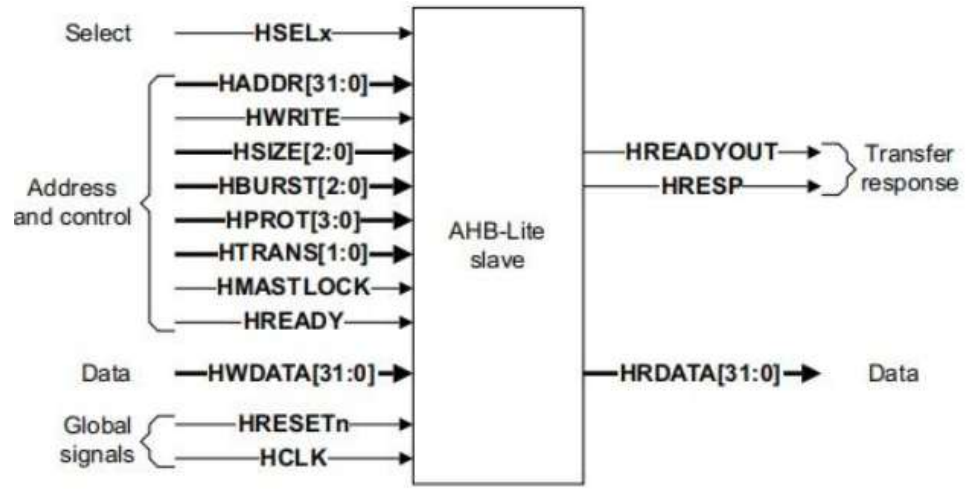
字段	含义
`AWLEN` / `ARLEN`	burst 长度减 1
`AWSIZE` / `ARSIZE`	每个 beat 的字节数，`2^SIZE`
`AWBURST` / `ARBURST`	地址类型：FIXED / INCR / WRAP
`WLAST` / `RLAST`	最后一个数据 beat 标志

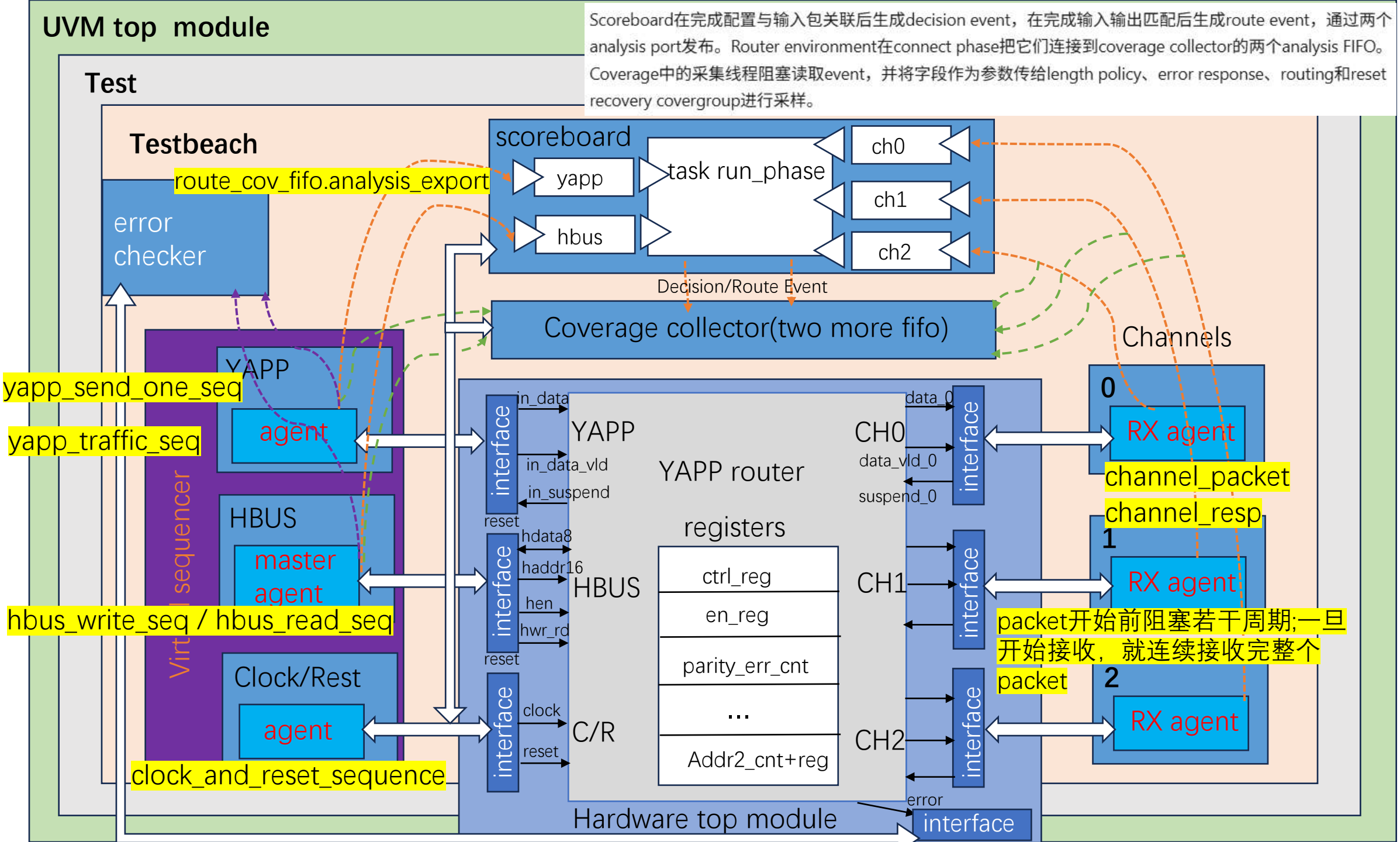
```
AWLEN = 0  → 1 beat
AWLEN = 3  → 4 beats
AWLEN = 15 → 16 beats
```

规则	含义
`VALID` 不能依赖 `READY`	发送方一旦准备好数据，就必须主动拉高 VALID，不能等对方 READY
`VALID=1 && READY=0`	当前信息必须保持稳定
`READY` 可以先拉高，也可以后拉高	接收方可随时表示是否能够接收
handshake 后可发送下一笔	同一个通道可以每拍完成一笔传输

`HSIZE[2:0]` 用于表示AHB每个beat的传输数据大小，实际传输字节数为 2^{HSIZE}，例如000表示1字节，001表示2字节，010表示4字节。






Scoreboard在完成配置与输入包关联后生成decision event，在完成输入输出匹配后生成route event，通过两个analysis port发布。Router environment在connect phase把它们连接到coverage collector的两个analysis FIFO。Coverage中的采集线程阻塞读取event，并将字段作为参数传给length policy、error response、routing和reset recovery covergroup进行采样。
UVM top module
Test
Testbeach
route_cov_fifo.analysis_export
error checker
scoreboard
yapp
hbus
task run_phase
ch0
ch1
ch2
Decision/Route Event
Coverage collector(two more fifo)
Channels
yapp_send_one_seq
yapp_traffic_seq
YAPP
agent
HBUS
master agent
hbus_write_seq / hbus_read_seq
Virtual sequencer
Clock/Rest
agent
clock_and_reset_sequence
interface
in_data
in_data_vld
in_suspend
reset
hdata8
haddr16
hen
hwr_rd
clock
reset
YAPP
HBUS
C/R
YAPP router
registers
ctrl_reg
en_reg
parity_err_cnt
...
Addr2_cnt+reg
CH0
CH1
CH2
data_0
data_vld_0
suspend_0
error
Hardware top module
0
RX agent
channel_packet
channel_resp
1
RX agent
packet开始前阻塞若干周期;一旦开始接收，就连续接收完整个packet
2
RX agent

第1页：项目概述

YAPP Router UVM Verification

项目项	内容
DUT功能	单输入、三输出Packet Router，带HBUS寄存器接口
输入协议	Header + Payload + Parity
路由规则	`addr 0/1/2 → Channel 0/1/2`
主要功能	路由、长度控制、错误处理、背压、寄存器、计数器、中断、Reset恢复
验证方法	Directed + Boundary + Constrained Random + Self-checking + Functional Coverage
环境结构	YAPP/HBUS/Channel/Clock-Reset VIP + Virtual Sequence + Scoreboard + Coverage + Error Checker

```
YAPP packet_delay
        ↓
发送Header/Payload/Parity
        ↓
Packet进入DUT目标FIFO
        ↓
DUT输出data_vld
        ↓
Channel生成resp_delay
        ↓
Driver保持suspend=1
        ↓
Monitor统计实际stall_cycles
```

这是一个单输入、三输出的可配置Packet Router UVM验证项目。输入Packet由Header、变长Payload和Parity组成，Header包含6 bit Length和2 bit Destination；HBUS负责配置最大包长、Router Enable、Counter和Interrupt。

环境包含YAPP输入Agent、HBUS Master Agent、三个Reactive Channel Agent和Clock/Reset Agent，通过Virtual Sequence协调配置、发包、输出响应、Backpressure和运行中Reset。

项目的重点是5路Analysis FIFO的配置感知型Scoreboard：它维护 `max_packet_size` 和 `router_en` 的Shadow State，根据输入Packet预测Forward/Drop及目标Channel，再比较Header、Payload和Parity。Coverage同时采集Monitor原始事务和Scoreboard关联后的Decision/Route Event。

我重点验证了三通道路由、N-1/N/N+1包长边界、非法地址、Bad Parity、Counter/Interrupt、Backpressure和Reset-in-packet；并定位了 `router_en` 判断嵌套错误导致Router关闭后合法Packet仍被转发的RTL/Spec Mismatch。

第3页：Verification Plan

Feature	Stimulus	Check	Coverage
基本路由	向addr 0/1/2发包	对应Channel输出，内容一致	`route × length`
长度边界	max=N，发送N−1/N/N+1	N−1、N转发；N+1丢弃	`max class × relation × action`
非法地址	addr=3	不应输出，错误计数/状态更新	`drop reason × action`
Bad parity	合法包注入错误parity	包继续转发，error/计数器更新	`parity × action`
HBUS寄存器	默认值、写入、读回	readback一致	`operation × register`
Counter/Interrupt	分别触发三类错误	counter递增、status置位、W1C清除	counter非零、status组合
Backpressure	不同Channel插入stall	无丢包、无乱序、输入可被反压	`length × stall`、`channel × stall`
Reset recovery	idle/in-packet时reset	旧包作废，新包正常路由	`reset context × recovery result`
Random regression	随机地址、长度、间隔、压力	Scoreboard自动检查	根据coverage hole补定向测试

面试问题

Channel VIP为什么reactive?

观察data_vld，生成suspend响应，模拟接收端背压

Monitor怎么知道stall?

`data_vld && suspend` 期间逐拍计数

Reset怎么处理旧包?

Monitor丢partial transaction；Scoreboard reset epoch

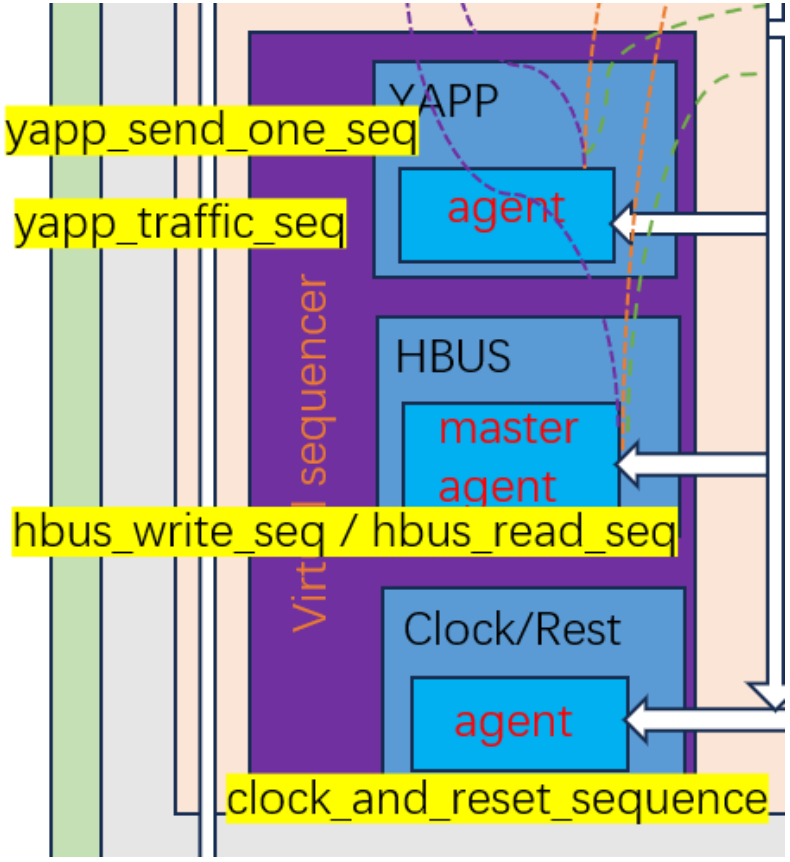
Error checker为什么独立?

Packet数据通路与sideband error职责分离

第4页：Stimulus组织

层级	负责什么	典型参数
Transaction	描述一个协议事务，默认约束合法	addr、length、payload、parity
VIP Sequence	产生基础协议流量或响应	packet count、gap、stall
Project Sequence	组合成Router专用流量	地址pattern、边界长度、错误类型
Virtual Sequence	协调多个VIP的时间关系	max size、target channel、pressure、reset时机
Test	选择VPlan场景并设置高层参数	使用哪个vseq、场景配置、checker开关

```
Test
  ↓ 选择场景和参数
Virtual Sequence
  ├─ YAPP traffic
  ├─ HBUS config/readback
  ├─ Channel response/backpressure
  └─ Clock/Reset
```



典型例子：长度边界测试

```
Test:
  max_size=20, target_channel=1

Virtual Sequence:
  1. HBUS写max_size=20
  2. 启动Channel 1 response
  3. 启动YAPP boundary sequence

Project Sequence:
  发送length=19、20、21

Scoreboard:
  19 → forward
  20 → forward
  21 → drop
```

随机测试

```
Virtual Sequence随机：
max_size、target_channel、pressure mode

Traffic Sequence随机：
packet length、gap、payload、parity概率

约束保证：
合法组合 + 受控错误注入 + 可复现seed
```

第一层：Raw Protocol Coverage 直接来自Monitor，证明接口上出现了什么：

```
YAPP Monitor    → addr、length、parity、payload pattern
HBUS Monitor    → read/write、register、readback data
Channel Monitor → output channel、length、实际stall cycles
```

第二层：Feature Coverage 来自Scoreboard关联后的语义事件，证明完整功能场景发生：

VPlan Feature	如何证明场景发生	对应Coverage
基本路由	addr0/1/2真实输入，对应Channel正确输出，覆盖不同包长	`packet_protocol_cg` + `routing_feature_cg`
长度策略	保存max-size快照，将Packet分类为below/equal/above，并确认forward/drop	`length_policy_cg`
非法地址/超长	记录drop reason，并确认action确实为drop	`error_response_cg`
Bad Parity	记录Bad Parity输入且Packet仍forward；Error由独立Checker检查	`packet_protocol_cg` + `error_response_cg`
HBUS寄存器	不同寄存器发生read/write，配置值被读回	`register_access_cg`
Counter/Interrupt	各Counter非零读回，Status组合出现，执行W1C	`counter_readback_cg` + `interrupt_status_cg`
Backpressure	每个Channel实际观察到不同stall强度，并覆盖不同包长	`flow_control_cg`
Reset Recovery	Reset发生在idle/in-packet，Reset后第一个Packet正确路由	`reset_recovery_cg`
Random Regression	不是独立Coverage Feature，用于补充上述Coverage Hole	使用全部Covergroup

```
Monitor证明“值出现过”
→ Scoreboard证明“输入、配置、输出属于同一个事件”
→ Feature Cross证明“VPlan场景完整发生”
→ Checker决定“这个事件是否正确”
```

跨接口Cross

```
HBUS Monitor更新配置镜像
        +
YAPP Monitor得到当前packet
        ↓
Scoreboard保存配置快照并计算length relation
        ↓
Decision Event
        ↓
同一个covergroup中sample并cross
```

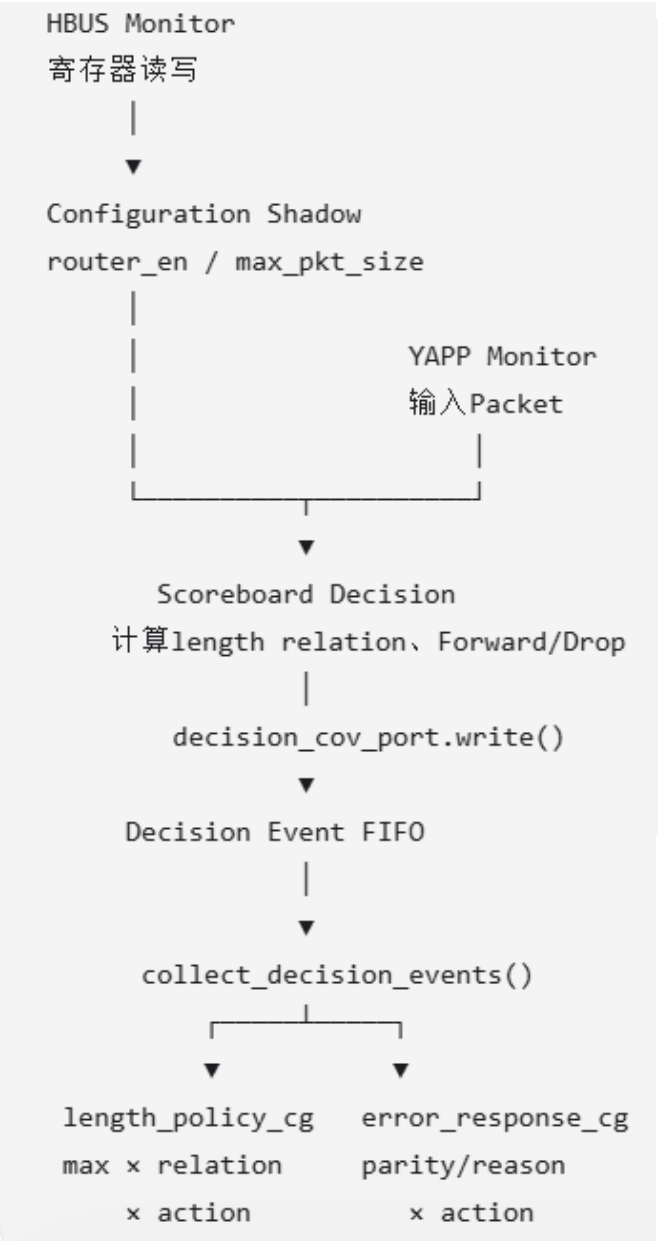
然后生成一个Decision Event：

```systemverilog
enable_at_accept
max_size_at_accept
length_relation
expected_action
```

Coverage Collector收到后调用：

```systemverilog
length_policy_cg.sample(
    enable_at_accept,
    max_size_at_accept,
    length_relation,
    expected_action
);
```

Feature Coverage Event Flow　Scoreboard关联配置、输入与输出，生成语义Event并驱动Covergroup采样





Decision Event

```
Input
addr, length, parity

Configuration Snapshot
router_en, max_pkt_size

Prediction
length_relation
expected_action
drop_reason
```

Route Event

```
Request
request_addr, packet_length

Response
expected_channel
actual_channel
compare_pass
```

一个Event可以服务多个Covergroup，但每次采样都由Collector显式调用，不是Event到达后自动采样。

Event产生	Event传输	Coverage采样
Scoreboard完成关联后创建Event	Analysis Port发送，Analysis FIFO缓存	Collector收到Event后显式调用 `sample()`
Decision发生在Forward/Drop预测之后	`write()` 与采样线程解耦	一条Event可以采样多个Covergroup
Route发生在输入输出匹配之后	`get()` 等待下一条Event	不相关场景通过条件控制，不盲目采样

Event不会自动触发全部Covergroup。Coverage Collector取得Event后，选择相关字段并显式调用对应的 `sample()`。

第5页：Scoreboard与Reset

Scoreboard输入

```
纯文本

YAPP FIFO        : 输入expected packet
HBUS FIFO        : 更新配置镜像
Channel 0/1/2 FIFO: 实际输出packet
```

```
HBUS写0x1000 → max_pktsize_reg
HBUS写0x1001 → router_enable_reg
```

```
取得YAPP输入包
→ 保存当前reset_epoch和配置快照
→ 判断forward/drop
→ 发布Decision Coverage Event
→ 合法包：等待对应Channel输出或Reset
→ 比较addr/length/payload/parity
→ 发布Route Coverage Event
```

Reset epoch

```
Reset前：packet_epoch = 2
Reset发生：reset_epoch = 3

packet_epoch != reset_epoch
→ 旧expected packet失效
→ 终止Channel FIFO等待
→ 不与Reset后的新包比较
```

对每个输入包保存当时的配置，避免后续HBUS写操作改变这个包的判断含义。

第7页：RTL必记提示

DUT结构

```
纯文本

host_ctl   : HBUS寄存器、计数器、中断
port_fsm   : 解析输入、路由、错误判断
3×FIFO     : 三个Channel输出缓存
```

FSM

State	行为
`HEADER_WAIT`	等待header，检查地址与长度，锁存目的Channel
`DATA_LOAD`	写入header/payload/parity，处理FIFO full和输入暂停
`DUMP_PKT`	非法地址或超长时，消费剩余输入但不写Channel FIFO

背压传播

```
纯文本

Channel suspend
→ FIFO停止读取
→ FIFO full
→ hold
→ in_suspend
→ YAPP Driver暂停发送
```

Rest(tb_bug)

我在增强Reset Recovery测试时，不再只等所有流量结束后复位，而是发送一个合法长包，并在Payload传输过程中插入Reset。仿真中出现了Reset后恢复包与Reset前Expected Packet错误匹配的问题。

定位后发现是环境里的两个并发问题：第一，Driver在Reset时会终止发送，但YAPP Monitor一旦采到Header，就会按照Header Length继续收Payload，原实现没有同时监听Reset，因此可能把Reset前Header和Reset后数据拼成一个Phantom Transaction；第二，Scoreboard可能已经从YAPP FIFO取出了Expected Packet，并阻塞等待对应Channel输出，这个Packet已经进入局部变量，单纯flush Analysis FIFO无法删除。

我把Monitor的Packet Collection和Reset Watcher并行运行，Reset先发生就丢弃Partial Transaction；同时在Scoreboard中增加Reset Epoch，等待Channel输出时并行监听Epoch变化，一旦Reset发生，就终止当前匹配、flush FIFO并恢复配置Shadow State。最后重新发送Reset后的恢复包，验证三通道路由和端到端比较恢复正常。

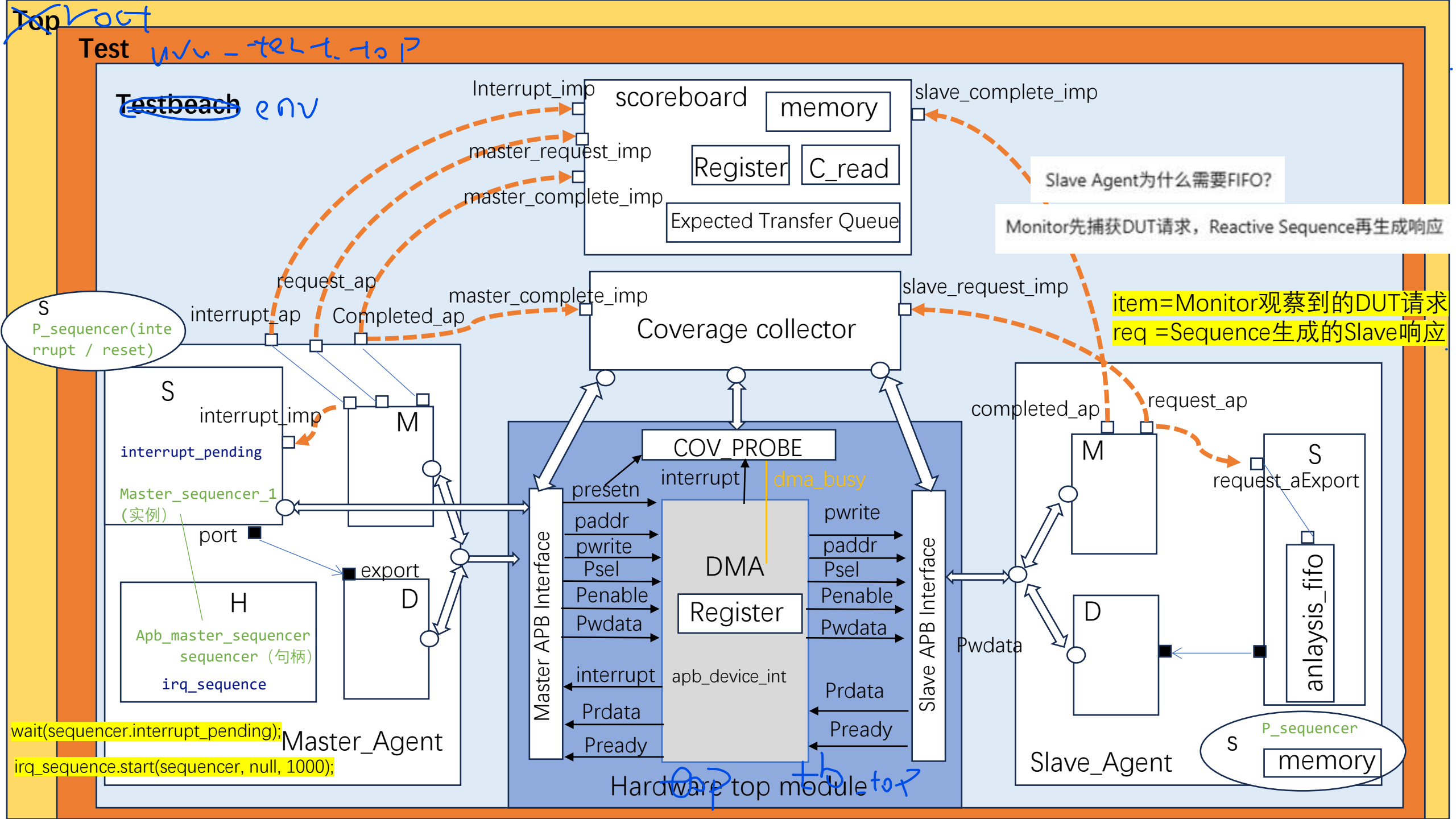
Dut_Bug

我通过HBUS将router_en置0，再向三个合法地址发送Packet。Scoreboard根据配置Shadow State预测Packet应全部丢弃仿真中YAPP Monitor确认三个合法Packet已经进入DUT，HBUS Monitor确认Enable配置已经写入，但三个Channel Monitor仍然分别收到了完整Packet，最终Output Analysis FIFO产生Unexpected Transaction。

我检查RTL后发现，`router_en==0` 的判断没有放在Header顶层决策路径，而是错误地嵌套在Length Error分支中。

因此合法Packet不会进入Length Error分支，也就完全绕过了 `router_en` 检查，最终仍被写入目标Channel FIFO。

`hw_top` 中的 `clock_and_reset_if` 直接连接全局Clock和Reset，`tb_top` 通过 `uvm_config_db` 把它的Virtual Interface句柄分别传给Scoreboard和Coverage Collector。Scoreboard通过Reset边沿推进 `reset_epoch` 、flush事务FIFO并恢复配置Shadow State；Coverage Collector则在Reset拉高时结合YAPP `in_data_vld` 记录Idle或In-packet Reset，并在第一个Reset后成功Route Event到来时采样Reset Recovery Coverage。整个过程不需要额外Reset FIFO。


Top
Test
Testbeach
env
Interrupt_imp
scoreboard
memory
slave_complete_imp
master_request_imp
Register
C_read
master_complete_imp
Expected Transfer Queue
Slave Agent为什么需要FIFO?
Monitor先捕获DUT请求，Reactive Sequence再生成响应
request_ap
master_complete_imp
slave_request_imp
item=Monitor观蔡到的DUT请求
req =Sequence生成的Slave响应
S
P_sequencer(interrupt / reset)
interrupt_ap
Completed_ap
Coverage collector
S
interrupt_imp
M
interrupt_pending
Master_sequencer_1
(实例)
port
export
H
Apb_master_sequencer
sequencer (句柄)
irq_sequence
D
COV_PROBE
interrupt
dma_busy
presetn
paddr
pwrite
Psel
Penable
Pwdata
interrupt
Prdata
Pready
Master APB Interface
DMA
Register
apb_device_int
pwrite
paddr
Psel
Penable
Pwdata
Prdata
Pready
Slave APB Interface
completed_ap
request_ap
M
S
request_aExport
D
Pwdata
anlaysis_fifo
wait(sequencer.interrupt_pending);
irq_sequence.start(sequencer, null, 1000);
Master_Agent
Hardware top module
Slave_Agent
s
P_sequencer
memory

这个项目验证的是一个单通道DMA控制器。DUT上游是APB Slave接口，由CPU侧配置SRC、DST、LEN并下发INIT或COPY命令；下游是APB Master接口，由DMA主动访问统一地址空间中的Memory。数据粒度固定为32 bit，COPY长度为N时，会产生N次Source Read和N次Destination Write。

验证环境采用UVM架构：Master Agent模拟CPU配置和Direct访问，Reactive Slave Agent模拟下游Memory并插入PREADY Wait-state；上下游Monitor把事务发送给Scoreboard和Coverage Collector。Scoreboard维护寄存器Shadow Model、Reference Memory及Expected Transaction Queue，对地址、方向、数据、传输顺序和Interrupt W1C行为进行端到端检查。

激励覆盖Direct、INIT、COPY、Interrupt、非法操作、地址Overlap、长度边界、Reset、Wait-state和Busy期间CPU访问，并通过Directed与Random Test、Seed复现和波形分析完成Regression与Debug。

`p_sequencer` 主要给业务Sequence提供类型化Sequencer句柄。Monitor检测IRQ后把pending、level和service count保存在Sequencer中，普通Sequence读取这些状态，等待中断处理完成后继续发送事务。Sequencer里保存的 `APB_VMINTF` 则主要服务于Reset测试，Sequence通过它观察时钟和PSEL/PENABLE状态，把presetn精确插入Idle、Setup或Access阶段。它不用于驱动IRQ，因为IRQ是DUT输出。

变量	谁修改	Sequence怎么使用
`interrupt_level`	Sequencer收到Monitor事件后更新	判断IRQ引脚是否仍为高
`interrupt_pending`	Sequencer置1，Handler处理后清0	判断是否存在待处理中断
`interrupt_count`	Sequencer统计	记录检测到多少次IRQ
`serviced_interrupt_count`	Handler处理完成后增加	Sequence等待ISR完成

`presetn` 负责复位DUT，`interrupt` 负责通知CPU侧处理中断，`dma_busy` 是内部运行状态；`COV_PROBE` 把这三个状态提供给Coverage Collector做白盒采样，本身不参与激励和结果检查。

第5页：核心验证计划

功能	如何测试	Scoreboard如何检查
Reset/Register	Reset后读写SRC、DST、LEN	检查默认值、写入和读回
Direct	Direct写后读，多种数据和地址	检查下游地址映射、方向、数据和上游读回
INIT	配置SRC/LEN，写INIT值	生成 `Write SRC+i` 队列，检查数据和数量
COPY	配置SRC/DST/LEN并预填Source	生成Read/Write交替队列，检查COPY数据
Interrupt	制造Done、Overlap、非法LEN和非法操作	检查INT读回、IRQ和W1C
Wait-state	延迟上下游PREADY	Scoreboard检查不丢不重，SVA检查信号稳定
Busy/Random	DMA期间CPU访问及随机组合	检查DMA队列和数据没有被破坏

COPY重点

```
Direct看端到端
INIT看连续写
COPY看先读后写
Interrupt看状态与清除
```

```
Expected Queue:
Read SRC+i
Write DST+i

COPY Read Data Queue:
保存实际Read Data
用于检查下一笔Write Data
```

Wait-state测试同时覆盖上下游：上游等待由DUT随机PREADY延迟产生，下游等待由Reactive Slave Sequence控制；Coverage统计等待周期，SVA检查上游信号稳定，Scoreboard检查等待后没有事务丢失、重复或数据错误。

Busy测试在启动DMA INIT后不等待完成，立即发起Direct、寄存器及非法CPU访问；通过内部busy probe确认请求确实落在DMA运行期间，再由Scoreboard检查原有DMA下游队列、地址和数据没有被破坏。

Random Test在合法安全边界内随机组合Direct、INIT、COPY、非法长度及Reserved访问，默认每个Seed执行40次；多Seed回归扩大状态组合，由Scoreboard持续检查DMA队列、Reference Memory和Interrupt状态，并支持通过Seed复现失败。

第4页：Stimulus组织

层级	负责什么	DMA项目中的实现/参数
Transaction	描述单笔APB事务	`addr、write、data、addr_delay、ready_delay`
Base Sequence	封装基础协议操作	`cpu_read/write、run_init、run_copy、reset注入`
Reactive Slave Sequence	根据DUT请求产生响应	Memory读写、`PRDATA`、`PREADY delay`
Project Sequence	组合DMA功能场景	Direct、INIT、COPY、Interrupt、Busy、Reset、Random
协同机制	协调Master、Slave和IRQ	Slave FIFO、`p_sequencer`状态、Interrupt Handler
Test	选择Sequence并配置场景	设置Master/Slave default sequence、Seed、Random次数

本项目未使用Virtual Sequence：Master负责CPU访问，Slave通过FIFO被动响应DUT请求，IRQ由独立Handler在Master Sequencer上启动高优先级Sequence。

DMA COPY地址关系测试

```
Test：
dma_apb_copy_test

Master Sequence：
dma_apb_copy_sequence

Slave Sequence：
apb_slave_basic_sequence
```

Stimulus流程

1. Direct Write预填Source Memory
2. 配置DMA_SRC、DMA_DST、DMA_LEN
3. 写DMA_COPY启动搬运
4. Slave Agent响应Source Read和Destination Write
5. 等待Done Interrupt及W1C清除

```
LEN：1、4、16、33
地址关系：
SRC = DST
前向/后向Overlap
前向/后向Adjacent
Disjoint
```

检查内容：

- Read/Write必须交替；
- Source/Destination地址递增正确；
- COPY写数据等于前一笔Read返回值；
- 下游传输数量等于`2 × LEN`；
- Reference Memory最终状态正确；
- Overlap和Done Interrupt符合预期。

第4页：Scoreboard重点

接口	来源	用途
`master_request_imp`	Master `request_ap`	Setup阶段提前建立预期
`master_complete_imp`	Master `completed_ap`	更新寄存器模型，检查上游结果
`slave_complete_imp`	Slave `completed_ap`	检查DUT实际下游事务
`interrupt_imp`	Master `interrupt_ap`	记录IRQ引脚变化

Scoreboard有四个输入。Master Request在上游Setup阶段到达，用于提前建立Direct下游预期，以及预测非法操作和COPY重叠；Master Complete用于更新SRC、DST、LEN寄存器模型，检查寄存器和Direct读回，并在INIT/COPY命令完成后生成预期下游队列；Slave Complete负责将DUT实际完成的下游事务与队列逐笔比较；Interrupt接口记录IRQ引脚状态。

核心检查链

```
Master Request/Complete
→ 生成Expected Slave Queue
→ Slave Complete提供Actual
→ 比较Direction、Address、Data、Order
```

内部模型

```
Register Model: SRC、DST、LEN
Reference Memory: 预测Memory内容
Expected Transfer Queue: 预期下游事务
COPY Read Data Queue: 保存COPY读回数据
Expected Interrupt: 预测4位中断状态
```

一、Coverage整体结构

建议把Coverage重新理解成三层：

覆盖层次	Covergroup	主要覆盖内容	对应验证计划
输入场景	`cg_master_access`	地址区域、读写方向、LEN边界、数据类型	Register、Direct、非法访问
输入场景	`cg_dma_command`	INIT/COPY、LEN分类、SRC/DST关系、配置是否完整	INIT、COPY、Overlap
中断行为	`cg_interrupt`	Done、Overlap、Invalid Length、Invalid Op、组合状态、W1C	Interrupt、异常场景
输出行为	`cg_slave_request`	DUT发起的下游读写、访问Source/Destination、写数据类型	Direct、INIT、COPY
输出行为	`cg_slave_completion`	实际完成的下游读写、地址目标和数据类型	Direct、INIT、COPY
时序状态	`cg_apb_wait`	上下游、读写方向、等待周期	Wait-state
时序状态	`cg_reset`	Idle、Setup、Access阶段Reset	Reset
内部状态	`cg_probe_state`	IRQ与DMA Busy的高低及跳变	Interrupt、Busy
组合压力	`cg_busy_attempt`	DMA Busy时CPU访问的[illegible]读写方向	Busy/Rando[illegible]

Hint

Transaction适合功能分类
VIF适合逐周期时序
COV_PROBE是可选白盒覆盖增强

Master事务告诉我“配置了什么、下了什么命令”；
Slave事务告诉我“DMA实际发出了什么访问”；
Virtual Interface告诉我“这个访问经历了什么时序”；
COV_PROBE告诉我“DUT当时处于什么内部状态”。

`COV_PROBE` 则把DUT内部 `is_dma_busy` 送给Coverage Collector，在CPU请求setup时采样：

`busy × 地址区域 × read/write`

证明 Lash场景确实发生在 `dma_busy=1` 期间，而不是DMA已经结束后才发送。

我的Functional Coverage不是简单按照接口信号罗列，而是从Verification Plan拆成三层。第一层是输入场景覆盖，通过Master完成事务采集地址区域、读写方向、LEN边界以及INIT/COPY命令上下文；其中Coverage Collector维护一份轻量级配置快照，用于在命令完成时计算SRC/DST的Equal、Overlap、Adjacent和Disjoint关系。
第二层是输出行为覆盖，采集DUT产生的下游Read/Write请求以及实际完成事务，确认Source Read、Destination Write和不同数据类型都被激励到。第三层是时序状态覆盖，通过上下游Virtual Interface统计Wait周期和Reset发生阶段，再通过 `cov_probe` 观察IRQ和DMA Busy跳变，以及Busy期间的CPU访问。
Bin主要按照功能边界划分，例如LEN分为0、1、2到15、16和大于16；Cross只覆盖有功能关联的组合，例如Command×Length、COPY Length×地址关系以及APB侧别×方向×Wait。Coverage负责证明场景被执行，正确性则由Scoreboard和SVA判断。

Tb_Bug

在Direct Read/Write测试中，我增加了上下游非对称wait-state场景：下游Reactive Slave立即拉高PREADY，而DUT上游PREADY存在较长延迟。仿真时Scoreboard报出Unexpected Downstream Transfer，但检查地址和数据都没有问题。

我随后在波形中对比上游setup、上游completion和下游completion三个时间点，发现DUT在收到上游请求后已经发起下游访问，下游事务可能先完成，而上游CPU事务要等自己的PREADY延迟结束后才完成。

当时Scoreboard只在收到 `master_complete` 后才生成下游期望，因此 `slave_complete` 先到达时，Expected Queue还是空的，就产生了误报。

最后我将Master Monitor拆成 `request_ap` 和 `completed_ap` 两条路径：在setup阶段通过 `request_ap` 提前把Direct访问加入Expected Queue，在completion阶段再通过 `completed_ap` 检查上游读回数据和寄存器状态。这样无论上下游谁先完成，都能正确匹配。

这个是DUT Bug吗?

> 不是，是Testbench的建模时点不正确。DUT允许上下游完成顺序不同，Scoreboard不能假设上游一定先完成。

激励：

下游PREADY快 + 上游PREADY慢

现象：

slave_complete先到

expected queue为空

→ Unexpected Transfer

定位：

比较双侧Monitor时间戳

→ DUT数据正确，Scoreboard预测太晚

解决：

```
master_request/setup → 提前生成下游期望
master_complete      → 检查上游结果
slave_complete       → 匹配并弹出期望
```

Dut_Bug

第二个问题是在DMA寄存器Readback测试中发现的。我先写入合法的LEN值，再读取DMA_LEN。寄存器返回值是正确的，但DUT的Interrupt却意外拉高，读取DMA_INT后发现 `invalid_length` 位被置位。

我在波形中确认当时访问地址是DMA_LEN， `PWRITE=0` ，说明它是一笔读事务；但Master Driver在读事务中将无意义的PWDATA驱动为0。继续追踪内部信号后发现，RTL的非法长度判断只检查了 `is_dma_len、PREADY和PWDATA` ，没有限定 `PWRITE` 。因此，即使执行的是读操作，只要PWDATA为0或大于16，也会错误地产生invalid-length中断。

根因是RTL让读事务中的don't-care PWDATA参与了功能判断。我提交的修复建议是在 `PSEL && PENABLE && PREADY && PWRITE` 的写完成条件下检查非法长度。RTL修复前，环境中临时让DMA_LEN读事务携带合法的PWDATA值，避免虚假中断，同时补充不同PWDATA取值下的LEN Read回归场景。

读事务的PWDATA为什么是0?

> APB读事务中PWDATA不参与传输，Driver通常驱动默认值0，也可能保留前一笔值。正因为它是don't-care，RTL不应该依赖它产生功能副作用。

DUT功能 | 地址空间

PADDR[31:28]	功能
0x0	Direct Access
0x1	SRC
0x2	DST
0x3	LEN
0x4	INIT
0x8	COPY
0xF	INT
其他	Invalid Operation

三条功能路径

合法LEN: 1~16
IRQ输出: |dma_int
清除方式: W1C

Direct: 上游访问直接转发到下游

INIT: 写SRC+i = 固定INIT数据

COPY: Read SRC+i, 再Write DST+i

COPY为什么两阶段？
APB一次只能完成一笔单方向事务，必须先读再写。

中断状态

Bit	含义
0	DMA Done
1	COPY Overlap
2	Invalid Length
3	Invalid Operation

第7页：仿真与回归

工具和文件

名称	作用
VCS / `simv`	编译和运行仿真
Exit Code	仿真进程是否正常结束
VDB	Coverage数据库
URG	合并VDB并生成Coverage报告
FSDB	Verdi波形文件
VPD	DVE波形文件

回归流程

VCS一次编译全部Test
运行定向Test
Random Test运行多个Seed
check_run.sh检查结果
URG合并VDB
生成HTML Coverage报告

Pass条件

Simulator exit code = 0
UVM_ERROR = 0
UVM_FATAL = 0
无Assertion Failure
Scoreboard队列和中断状态全部清空

引入RAL后可以规范描述：

- `DMA_SRC`：28位R/W；
- `DMA_DST`：28位R/W；
- `DMA_LEN`：5位R/W；
- `DMA_INT`：W1C；
- Reset值和字段访问权限；
- Frontdoor寄存器Sequence；
- Mirror和Readback检查。

典型结构：

RAL Sequence
→ Register Model
→ APB Adapter
→ Master Sequencer/Driver
→ DUT

Master Monitor
→ RAL Predictor
→ 更新Mirror

Shell脚本更适合环境配置、流程执行和回归控制；Makefile更适合定义编译、运行、覆盖率等目标并管理依赖。二者并非必须同时使用，小项目用Shell即可，较完整的验证环境通常以Makefile作为统一入口，再调用Shell或Python脚本完成具体任务。